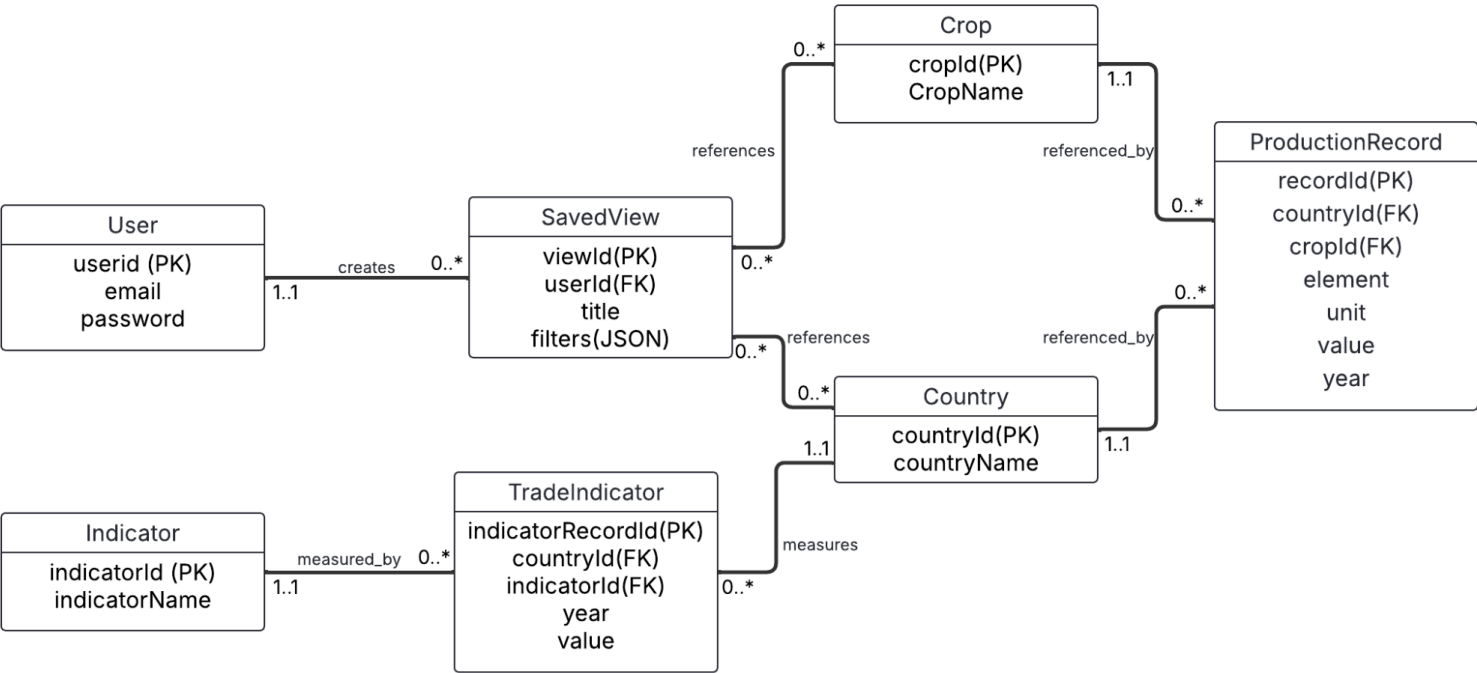


Oasis Intelligence — Database Design Document

UML Diagram



Assumptions, cardinality, and modeling rationale

The user entity represents the account of a user. Saved views is an entity rather than an attribute in users because a single user creates zero to many views according to their needs. When a user generates a view, it is only visible to them, therefore every view must have a one to one relation with a user. Each view can reference zero to many crops and/or countries in its JSON attribute which allows users to create their own visualizations and dashboard for analytics.

Crops is its own entity instead of being an attribute in production records because crop identity is shared among different tables. Not all crops have production records in a specific element. This will reduce redundancy and make creating views on a specific crop easier. A crop can be referenced by zero to many saved views. And a crop can be harvested by 0 or many countries, therefore it has a zero to many relation with production records.

Country is its own entity because it is referenced by multiple other tables. This reduces redundancy and makes updates(adding more countries in the future) and creating views easier. A country can be referenced by zero to many views. And a country can grow multiple or no crops, therefore it has a zero to many relation with production records. Each country can be mentioned by zero to many rows in trade indicators.

Production records are a relation between crops and countries. It's a separate entity because it cannot be part of either crop or country table as an attribute because there is a lot of information being repeated in doing so, and not all crops and countries have production records. Each production record can only reference one country and one crop for each unique element attribute. Element describes the crop specific attributes such as "area harvested" or "volume", etc. Therefore, production records have one to one relation with crops and country.

Indicators is a separate entity, not an attribute of trade indicators because there are multiple recurring indicators such as Irrigated land %, food import dependency ratio, etc. Putting them into their own table avoids repeating indicator's description in every trade indicator row. Each of the indicators can be referenced by trade indicators zero to many times.

Trade indicators represent events/observations and measure multiple different indicators for different countries. It wouldn't make sense to attach these rows to the country table as an attribute because there are a lot of repetitions, and not all countries have a trade indicator. Each trade indicator row can only mention one country making it a one to one relation.

Normalization Explanation

Our database schema follows **Third Normal Form (3NF)**. The main goal of our normalization process was to reduce repeated data, separate independent concepts into their own entities, and make sure that non-key attributes depend only on the primary key of their table.

The schema is in **First Normal Form (1NF)** because each table has a primary key, and each column contains atomic values rather than repeated groups. For example, the Country table stores one country per row, the Crop table stores one crop per row, and the Indicator table stores one indicator per row. The production and trade indicator data are stored as separate records rather than placing multiple years or multiple crop values into a single field.

The schema is in **Second Normal Form (2NF)** because every non-key attribute depends on the full primary key of its table. Most tables use a single-column primary key, such as countryId, cropId, indicatorId, recordId, and viewId, so there are no partial dependencies. In ProductionRecord, attributes such as element, unit, value, and year describe the specific production record identified by recordId. In TradeIndicator, attributes such as year and value describe the specific indicator observation identified by indicatorRecordId.

The schema is in **Third Normal Form (3NF)** because there are no transitive dependencies between non-key attributes. For example, countryName is stored only in the Country table instead of being repeated in every ProductionRecord or TradeIndicator row. Similarly, cropName is stored only in the Crop table, and indicatorName is stored only in the Indicator table. The production and trade indicator tables store foreign keys such as countryId, cropId, and indicatorId instead of repeating descriptive information. This prevents update anomalies and keeps the database consistent if a country, crop, or indicator name needs to be changed.

The SavedView table is also normalized because it stores user-created dashboard configurations separately from the Usertable. A user can create zero or many saved views, so saved views should not be stored as attributes inside the user record. Each saved view has its own viewId, references one user through userId, and stores the selected dashboard filters. The original public datasets remain read-only, while user-created saved views are stored separately.

Overall, the schema satisfies 3NF because each entity represents one main concept, repeated values are separated into lookup/reference tables, and all non-key attributes depend only on the key of their own table. This design avoids unnecessary duplication and supports efficient searching, filtering, and visualization across countries, crops, production records, and trade indicators.

Logical Design:

Users(userId:INT [PK], email:VARCHAR(255), password:VARCHAR(255))

SavedViews(viewId:INT [PK], userId:INT [FK to Users.userId], title:VARCHAR(255), filters:JSON)

Crops(cropId:INT [PK], cropName:VARCHAR(255))

Countries(countryId:INT [PK], countryName:VARCHAR(255))

ProductionRecords(recordId:INT [PK], countryId:INT [FK to Countries.countryId], cropId:INT [FK to crops.cropId], element:VARCHAR(255), unit:VARCHAR(20), value:INT, year:INT)

Indicators(indicatorId:INT [PK], indicatorName:VARCHAR(255))

TradeIndicators(indicatorRecordId:INT [PK], countryId:INT [FK to countries.countryId], indicatorId:INT [FK to indicators.indicatorId], year:INT, value:INT)